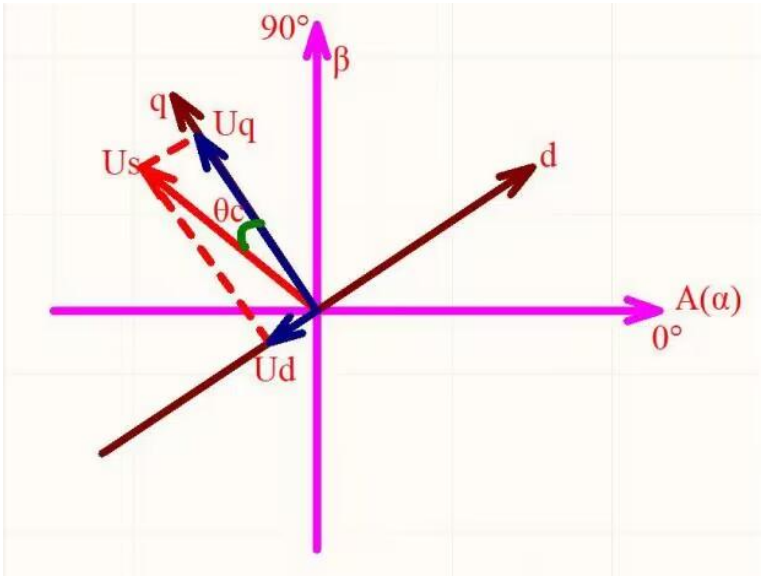
58-FOC 11

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。
上节课最后讲了旋转的角度需要做表。

θ(i)			
45		8192	8192
26.5651		4836.02	4836
14.0362		2555.22	2555
7.12502		1297.07	1297
3.57633		651.052	651
1.78991		325.843	326
0.89517		162.961	163
0.44761		81.4857	81
0.22381		40.7435	41
0.11191		20.3718	20
0.05595		10.1859	10
0.02798		5.09296	5
0.01399		2.54648	3
0.00699	99.7711	1.27324	1

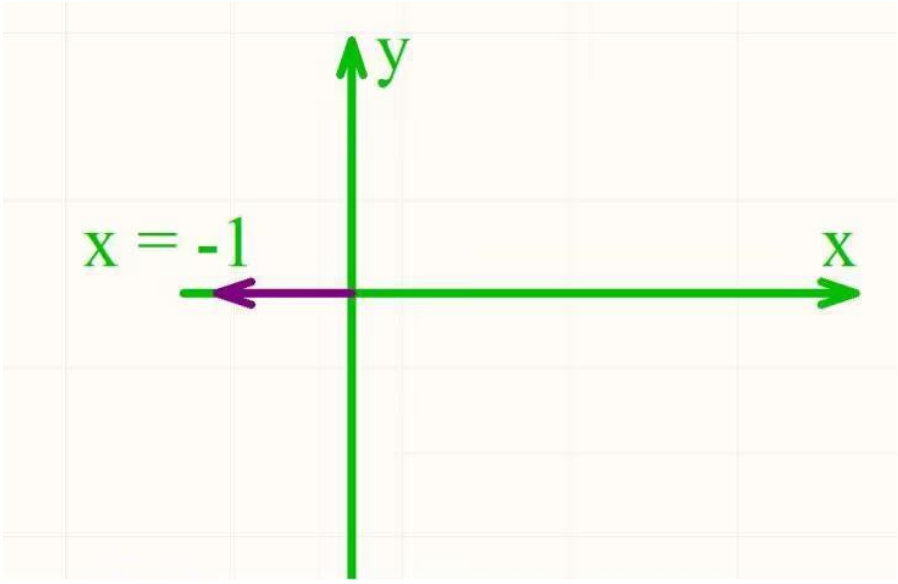
大家看，转换好之后，最右边的一列就是角度对应的 Q15 格式的值。那么，这一列数据就是我们编程的时候要使用的数据。好，那我们再来看另一个问题啊。



问：我们应该希望 Ud 分量为 0 是吧，那么是不是实际的 θ 很小是吧？
答：不是，Id=0 时，Ud<0 的，因为电压是超前电流的。

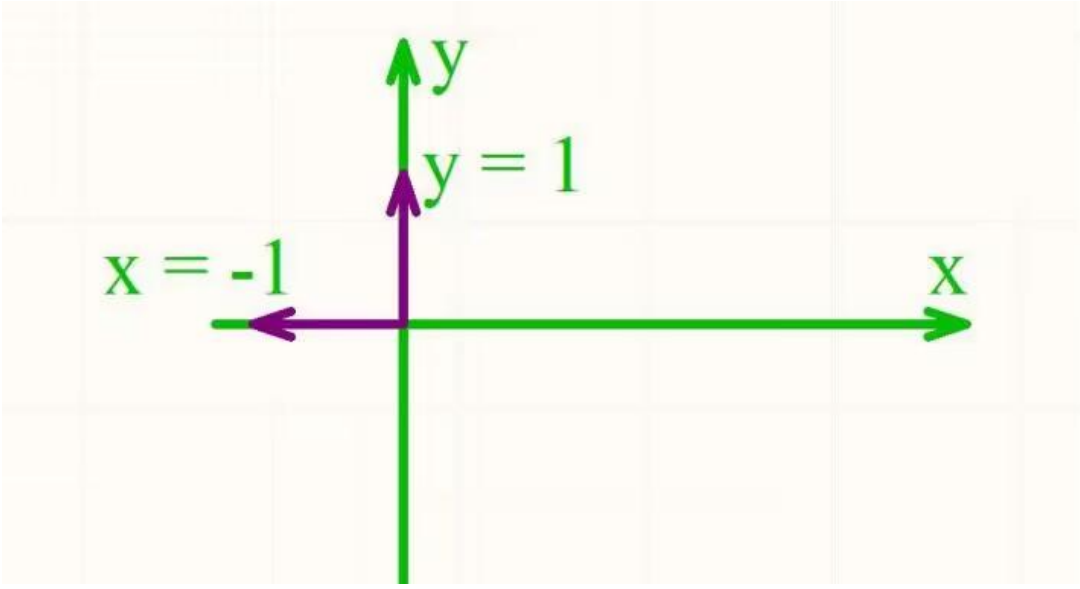
我们知道，电流环的输出是 ud 跟 Uq 对吧？如果说矢量的 Us 最大模长是 100% 。也就是 1 的话，从目前的图来看，应该 Ud 是达不到-1，而 Uq 也达不到 1 的对吧？如果是另一个方向旋转，则 Ud 达不到-1，Uq 也达不到-1。所以，正常情况下，应该 Ud Uq 都不会达到最大值。就是说，正常情况下，没有异常值。但是异常情况呢，我们也要考虑一下，如果出现异常，怎么去处理？异常值的话，无非就是输入的值不符合实际的情况。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



问：X=-1　是不是表示 Us 为 180 度吗？
答：是的，（-1，0）表示 Us 为 180 度。

大家看，如果输入这种情况，x=-1，如果我们顺时针旋转的，是不是 y 坐标系要变为 1 啊？



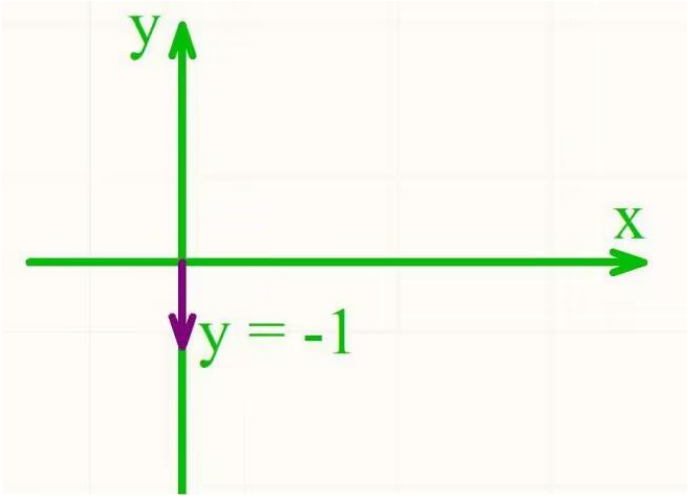
大家看，这样旋转的话 y 坐标变为了 1。这个 1 如果我们用 Q15 去表示的话-1 对应 32768。把-32768 取反加 1 之后就是 32768。此时呢，实际上这个 32768 已经超过了 16 位有符号数表示的最大范围了。16 位有符号数，正的最大值是 32767。而我们旋转之后，对应得到的是 32768，这样的话就可能导致计算结果异常。

问：应该转到（-1,1）吧？
答：这是 45 度，改变象限我们是希望转 90 度。

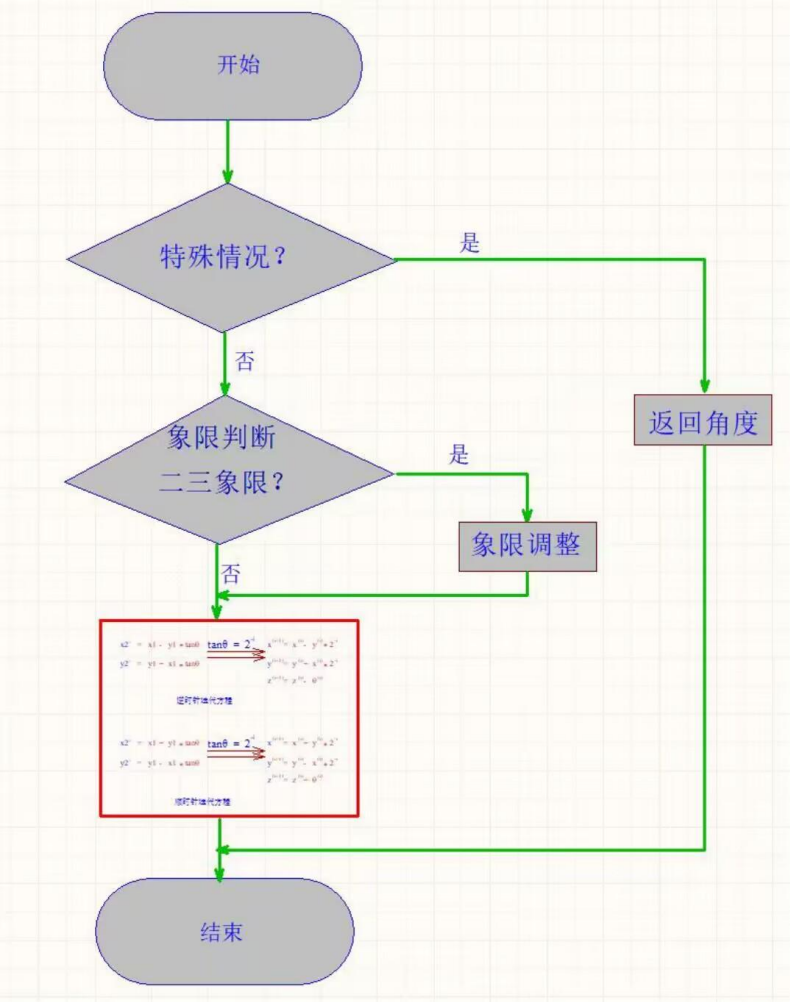
所以啊，我们在实际使用的时候，可以把这种情况特殊处理一下。处理方法呢，就是我们判断到 x=-1 的时候，就可以设定一个初始角度是 180°，而这样处理之后，就相当于是旋

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

转了 180°，这一次旋转呢，其实就把矢量旋转到了 x 的正半轴 。这样的话，后续的迭代都不需要计算了，就是说，我们碰到这种特殊情况后，就让算法结束。



那么，同理呢，实际上也不会存在 y=-1 的情况，如果出现这种情况，我们也可以返回一个特定的角度-90°，同样呢，这个特殊情况呢，我们也让算法结束。所以，我们目前的算法，可以增加这两种特殊条件的判断，如果判断出来，就直接返回相应的角度，结束算法，就不再去迭代计算了。



这个时候呢，我们可以再完善一下刚才的流程图。也就是说，整个算法一进来，先判断特殊情况，如果遇到特殊情况，直接返回对应的角度，整个算法结束。如果不属于特殊情况，

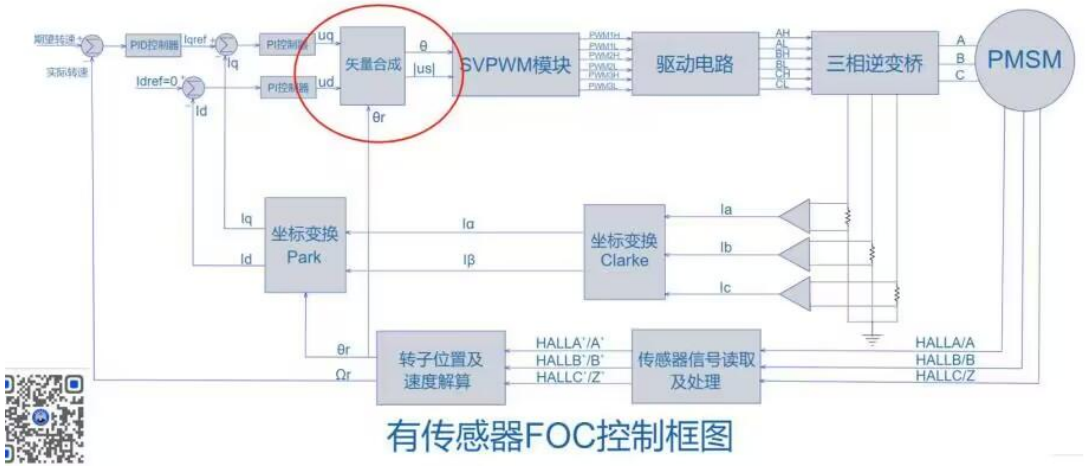
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

再去判断象限，最后呢，再进行迭代计算，得到对应的角度。好，那么到现在，整个 Cordic 数字迭代算法的思路，我们就全部分析完了。

后面呢就剩下编程来实现对应的功能了。

问：这里这个函数输入的，x,y 坐标怎么限定他们的范围？不能随便取很大的数据吧？
答：使用 Q15 就可以，由 Q15 定义就可以了。

问：这种求法的误差是多少度？
答：精度取决于迭代次数，是可控的。



有传感器FOC控制框图

那么，这个时候我们再回过头来看下这幅框图。

问：第一次迭代，x,y 会不会迭代出超过 1 的数据？
答：不会，我们只关心迭代角度的变化，不关心其幅值变化。

电流环的输入是 Ud 跟 Uq，根据这两个值呢，我们可以把|Us|求解出来。同样根据 Ud Uq 可以把超前角 θc 求解出来，然后根据角度关系，

θ = θr + 90° + θc θ = θr - 90° - θc。

只不过，求解超前角的时候呢，我们要用到一个反正切函数，θc = arctan(-Ud/Uq)。而我们单片机如果没有三角函数或者反三角函数计算的模块，所以呢要实现反正切的话，就要自己编写算法。此时呢，我们就采用了迭代的思想，使用了 Cordic 数字迭代算法来实现反正切的功能。这样的话，对应的超前角也就可以求解出来了，自然呢，θ 也就求解出来了。

所以，SVPWM 模块的两个输入量就确定下来了，确定下来之后，SVPWM 模块就可以输出对应的调制波，来控制对应的三相逆变桥，从而驱动电机去运转。那么，到现在为止，整幅控制框图对应的内容，我们就分析一遍了。后面呢，我们就把这部分对应的代码分析一下，然后就进入无感 FOC 部分了啊。

那么，到此为止，我们把带霍尔传感器的 FOC 控制框图涉及到的内容分析完了。当然传感器实际应用中也不一定是霍尔传感器，也可以是其它传感器，道理都是一样的。从框图的宏观层面，再到框图中各个模块的内部，都讲了一遍。现在我们就开始分析这部分的代码实现啊。

问：θc 可不可以等于 0？
答：可以，这时 Id>0， 处于增磁控制状态。

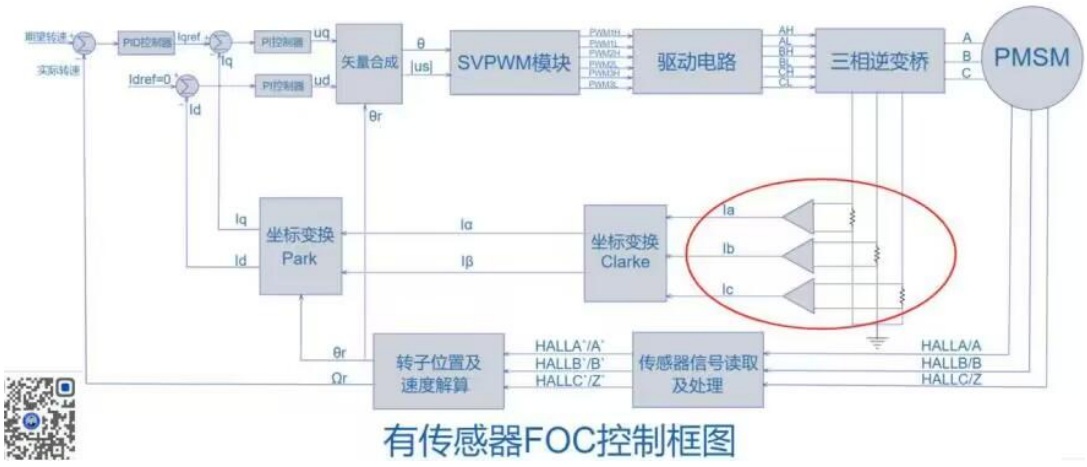
本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

分析的代码呢，已经提前上传到群文件了，大家可以下载后对照学习。



我们分析代码呢，先结合框图，每个模块对应的代码分析一遍后，再汇总。

问：求 θ c 的目的是为了不增磁或不弱磁吗？
答：我们的课程的控制目标是 Id=0 控制。



因为我们要对电流进行控制，所以比较重要的部分就是要采集三相相电流。所以，我们就需要采集电流电阻两端的电压值，然后再处理得到能够反映电机相电流正负的值。

那么，这里大家是否还记得电流的具体的处理方式啊?

我们在前面总结过关于相电流的采集处理：

第一、初始时刻，采样运放输出值，得到包含偏置电压的采样值

第二、电机运行时，正常采样运放的输出值

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

第三、用初始时刻的运放输出值，减去电机运行时的运放输出值，得到能够反映相电流方向及大小的值。所以呢，我们第一步是要采集电机没有运行时的偏置电压值。这个采集呢，我们可以在初始化阶段来完成。

```
void StartProgram(void)
{
    unsigned int Temp;

    Initialize();
    tSysClock.StartupDelay = 1000; //1s
    ENIE();

    do
    {
        DISIE();
        Temp = tSysClock.StartupDelay;
        ENIE();
    }while(Temp != 0);

    GetCurrentOffset();

    while(1)
    {
        FocApp();
        ProtectApp();
        LedApp(tProtect.Code);
    }
}
```

大家来看下这段代码，在初始化函数执行结束后，我们给一个计数变量设定了一个初值。接着使能 ADC 中断，那么，这个计数变量就会在 AD 中断中以 1ms 的周期来递减。从 1000 递减到 0，对应的时间也就是 1s。而在这 1s 的时间内呢，我们可以读取三相电流对应的运放输出偏置电压值。并且呢，为了让数据稳定些，在这 1s 的时间内，我们可以对读取到的数据做一个平均滤波处理。那么，当 1s 的时间到了后，我们就可以读取平均滤波值作为电压偏置值使用了。

```
            LedApp(tProtect.Code);
        }
    }
    /*==========================================================================*/
    //50us Interrupt
    void __attribute__((section(".main_section"),interrupt,shadow,no_auto_psv)) _AD1Interrupt(void)
    {
        IFS0bits.AD1IF = 0;

        AdcSample();
        GetCurrent();
        Observer();
        CurrentLoop();
        StatusControl();
        CalSinCos(tFoc.Eangle_Rotor);
        AdcFilter();
        SpiTransmit();
        DownCounter();
    }
    /*==========================================================================*/
```

1s 等待的过程中，中断一直在进行。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
void StartProgram(void)
{
    unsigned int Temp;

    Initialize();
    tSysClock.StartupDelay = 1000; //1s
    ENIE();

    do
    {
        DISIE();
        Temp = tSysClock.StartupDelay;
        ENIE();
    }while(Temp != 0);

    GetCurrentOffset();

    while(1)
    {
```

偏置值的获取，就是靠这个函数，那么，这样的话，就完成了获取三相电流值的第一步。好，那么整体的思路理解之后，我们来分析下这部分的代码。

问：采样和滤波隔这么远吗，滤波函数能放采样下面吗？

答：可以的，因为执行滤波函数的周期和采样周期是一致的。实时性要求高的，会考虑放在前面。

```
void StartProgram(void)
{
    unsigned int Temp;

    Initialize();
    tSysClock.StartupDelay = 1000; //1s
    ENIE();

    do
    {
        DISIE();
        Temp = tSysClock.StartupDelay;
        ENIE();
    }while(Temp != 0);

    GetCurrentOffset();
```

首先，红框中两句话执行完毕之后，计数初值以及 AD 中断就设定好了。如果没有满足中断进入条件的话，就会执行 do 循环。如果满足了中断进入条件的话，就会进行中断服务函数。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
//50us Interrupt
void __attribute__((section(".main_section"), interrupt, shadow, no_auto_psv)) _AD1Interrupt(void)
{
    IFS0bits.AD1IF = 0;

    AdcSample();
    GetCurrent();
    CurrentLoop();
    StatusControl();
    CalSinCos(tFoc.Eangle_Rotor);
    AdcFilter();
    SpiTransmit();
    DownCounter();
}
/*=====================================================================*/
static void DownCounter(void)
{
    tSysClock.PwmTick ++;

    if(tSysClock.PwmTick >= 20)
    {
        tSysClock.PwmTick = 0;
        if(tSysClock.StartupDelay != 0) tSysClock.StartupDelay --;
        if(tSysClock.PwmCountMs != 0) tSysClock.PwmCountMs --;
        if(tSysClock.LedCountMs != 0) tSysClock.LedCountMs --;
        if(tSysClock.ProtectCountMs != 0) tSysClock.ProtectCountMs --;
    }
}
/*=====================版权归张飞实战电子所有，违者必究！！！=====================*/
```

大家看，一旦进入中断服务函数，则就会读取 ADC 模块的采样转换值。

问：这里不是等中断吧 tSysClock.StartupDelay 减到 0 么？
答：这是等待滤波和系统稳定的时间。

并且呢，也有对应的计数变量去计数；读取采样转换值的话，执行的函数就是 AdcSample()。

```
/*=====================================================================*/
void AdcSample(void)
{
    if(tAdc.Index == 3)
    {
        tAdc.An3.ValueInstant = ADC1BUF0;
        tAdc.An0.ValueInstant = ADC1BUF1;
        tAdc.An1.ValueInstant = ADC1BUF2;
        tAdc.An2.ValueInstant = ADC1BUF3;

        AD1CHS0 = 0x0004;
        tAdc.Index = 4;
    }
    else if(tAdc.Index == 4)
    {
        tAdc.An4.ValueInstant = ADC1BUF0;
        tAdc.An0.ValueInstant = ADC1BUF1;
        tAdc.An1.ValueInstant = ADC1BUF2;
        tAdc.An2.ValueInstant = ADC1BUF3;

        AD1CHS0 = 0x0003;
        tAdc.Index = 3;
    }
    else
    {
        tAdc.An3.ValueInstant = ADC1BUF0;
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

这个函数的话，大家比较熟悉了，CH0 呢是在轮询切换采样的通道。CH1~CH3 呢，是固定的对应三相相电流的采样通道。根据这个函数呢，我们可以读取到初始时刻，运放的输出值的瞬时值。要得到平均值的话，就要把多个瞬时值累加起来，做一个平均。

```
/*=====*/
//50us Interrupt
void __attribute__((section(".main_section"), interrupt, shadow, no_auto_psv)) _AD1Interrupt(void)
{
    IFS0bits.AD1IF = 0;

    AdcSample();
    GetCurrent();
    CurrentLoop();
    StatusControl();
    CalSinCos(tFoc.Eangle_Rotor);
    AdcFilter();
    SpiTransmit();
    DownCounter();
}
/*=====*/
```

滤波的话，就是使用这个滤波函数。

```
/*=====*/
void AdcFilter(void)
{
    if(tSysClock.StartupDelay != 0)
    {
        tAdc.An0.ValueSum += tAdc.An0.ValueInstant;
        tAdc.An0.ValueNum ++;
        if(tAdc.An0.ValueNum >= 64)
        {
            tAdc.An0.ValueAverage = tAdc.An0.ValueSum >> 6;
            tAdc.An0.ValueSum = 0;
            tAdc.An0.ValueNum = 0;
        }

        tAdc.An1.ValueSum += tAdc.An1.ValueInstant;
        tAdc.An1.ValueNum ++;
        if(tAdc.An1.ValueNum >= 64)
        {
            tAdc.An1.ValueAverage = tAdc.An1.ValueSum >> 6;
            tAdc.An1.ValueSum = 0;
            tAdc.An1.ValueNum = 0;
        }

        tAdc.An2.ValueSum += tAdc.An2.ValueInstant;
        tAdc.An2.ValueNum ++;
        if(tAdc.An2.ValueNum >= 64)
        {
            tAdc.An2.ValueAverage = tAdc.An2.ValueSum >> 6;
            tAdc.An2.ValueSum = 0;
            tAdc.An2.ValueNum = 0;
        }
    }
```

大家看下这个函数，因为这个偏置值我们仅在每次上电时读取。所以呢，整个求取平均值的代码，我们都是在初始化计数变量没有递减为 0 的时候执行的。这里是选取 64 次平均，得到对应的平均值。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

tAdc_Type 结构体里的变量有各个通道的结构体和电流偏移值。

```
/*==========================================*/
typedef struct
{
    unsigned int ValueInstant;
    unsigned int ValueAverage;
    unsigned int ValueSum;
    unsigned int ValueNum;
    unsigned int ValueFilter;
}tAnx_Type;
/*==========================================*/
typedef struct
{
    tAnx_Type An0,An1,An2,An3,An4,An6;
    unsigned int Index;
    unsigned int IoffsetA;
    unsigned int IoffsetB;
    unsigned int IoffsetC;
}tAdc_Type;
```

```
    CalSinCos(tPoc.tAngle_Rotor);
    AdcFilter();
    SpiTransmit();
    DownCounter();
}
/*==========================================*/
static void DownCounter(void)
{
    tSysClock.PwmTick ++;

    if(tSysClock.PwmTick >= 20)
    {
      tSysClock.PwmTick = 0;
      if(tSysClock.StartupDelay != 0) tSysClock.StartupDelay --;
      if(tSysClock.PwmCountMs != 0) tSysClock.PwmCountMs --;
      if(tSysClock.LedCountMs != 0) tSysClock.LedCountMs --;
      if(tSysClock.ProtectCountMs != 0) tSysClock.ProtectCountMs --;
    }
}
```

接着呢，大家再来看下，中断函数中调用了一个倒计数函数。

问：这里可以改进一下，取 128 次，前 64 次去掉，取后 64 次取平均可能会更准？
答：这里取的是 1s 内的最后一个 64 哦，1s 内有很多个 64 次。

因为中断函数是 50us 执行一次，所以呢 PwmTick 这个变量也就是每 50us 递增一次。
所以，1ms 的话，也就是 PwmTick 变量递增了 20 次。所以，对应的 StartupDelay 变量就是 1ms 递减一次，初值设定 1000 的话，递减到 0 就对应 1s 的时间。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
void StartProgram(void)
{
    unsigned int Temp;

    Initialize();
    tSysClock.StartupDelay = 1000; //1s
    ENIE();

    do
    {
        DISIE();
        Temp = tSysClock.StartupDelay;
        ENIE();
    }while(Temp != 0);

    GetCurrentOffset();

    while(1)
    {
        FocApp();
```

所以，这段程序设定了计数初值，并使能中断，如果满足中断进入条件，就会进入中断执行中断任务。如果不满足中断进入条件，就会执行 do 循环。那么，因为进入中断后，上电延时变量会递减。所以，当延时变量递减为 0 之后，就会跳出 do 循环。

```
/*
void StartProgram(void)
{
    unsigned int Temp;

    Initialize();
    tSysClock.StartupDelay = 1000; //1s
    ENIE();

    do
    {
        DISIE();
        Temp = tSysClock.StartupDelay;
        ENIE();
    }while(Temp != 0);

    GetCurrentOffset();

    while(1)
    {
        FocApp();
        ProtectApp();
        LedApp(tProtect.Code);
    }
}
```

那么，跳出 do 循环之后呢，也就说明上电延时结束了，对应的 1s 也就结束了。此时呢，我们就可以读取平均滤波值，作为电压偏置值。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*==========================================================================*/
void GetCurrentOffset(void)
{
    tAdc.IoffsetA = tAdc.An0.ValueAverage;
    tAdc.IoffsetB = tAdc.An1.ValueAverage;
    tAdc.IoffsetC = tAdc.An2.ValueAverage;
}
```

这是对应的函数内容

再次总结一下

第一、初始时刻，采样运放输出值，得到包含偏置电压的采样值

第二、电机运行时，正常采样运放的输出值

第三、用初始时刻的运放输出值，减去电机运行时的运放输出值，得到能够反映相电流方向及大小的值

所以呢，这样我们就完成了获取相电流的第一步，得到偏置电压的采样值

好，我们再来看第二步

电机运行时，要获取运放的实时输出值。其实，这个步骤的话，就是直接获取三相相电流的瞬时采样值。

```
void AdcSample(void)
{
    if(tAdc.Index == 3)
    {
        tAdc.An3.ValueInstant = ADC1BUF0;
        tAdc.An0.ValueInstant = ADC1BUF1;
        tAdc.An1.ValueInstant = ADC1BUF2;
        tAdc.An2.ValueInstant = ADC1BUF3;

        AD1CHS0 = 0x0004;
        tAdc.Index = 4;
    }
    else if(tAdc.Index == 4)
```

也就是说，我们只需要使用这三个瞬时值就行。

第三、用初始时刻的运放输出值，减去电机运行时的运放输出值，得到能够反映相电流方向及大小的值

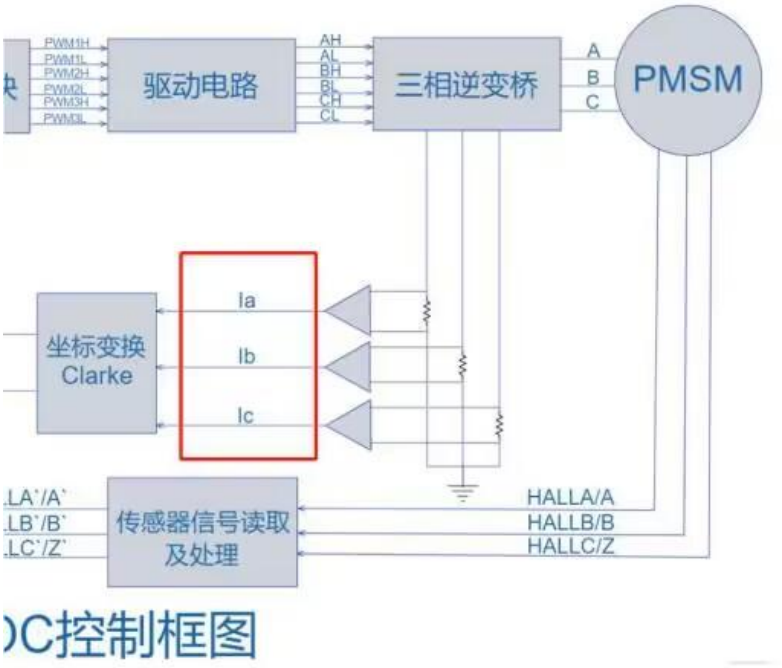
然后呢，第三步为了要得到能够反映相电流方向及大小的值，就要用初始偏置值减去瞬时值。

```
/*==========================================================================*/
static void GetIabc(void)
{
    signed int sIa,sIb,sIc;

    sIa = (signed int)((tAdc.IoffsetA - tAdc.An0.ValueInstant) << 5);
    sIb = (signed int)((tAdc.IoffsetB - tAdc.An1.ValueInstant) << 5);
    sIc = (signed int)((tAdc.IoffsetC - tAdc.An2.ValueInstant) << 5);
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

这个时候大家来看下这个函数，根据第三步的描述，我们就使用偏置值，减去瞬时值。那么，这里为什么要对差值左移 5 位呢？ 我们要把 AD 采样值转换为 Q 格式的数值，而怕数据溢出导致结果异常，所以我们只左移了 5 位。这样的话，也就得到了归一化后的能够反映相电流大小及方向的值。



问：这里都左移 5 位了，后面处理是不是还要缩小回来？

答：不需要，因为控制的目标值电流也是归一化处理的，电压也是，到处都归一化了，本质上只要目标电流跟上给定电流，什么单位也就不重要了。

问：这归一化处理算法有没有什么通用的原则？

答：结合实际运行电流大小，和需要测量的电流范围，合理选择归一化电流。

那么，代码分析到这里，理论上我们就得到了框图中的 Ia，Ib，Ic。根据这三个值呢，去计算坐标变换，就可以得到 Iα 及 Iβ 。大家课后看一下程序代码，下节课接着讲。

谢谢大家，晚安！

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。